MIN-HASHING AND LSH

Name: Manisha Bhalla
Roll No: M25DE1050
Github: https://github.com/Manisshabhlla/Min-hashing-and-LSH

Question 1: Create k-Grams: Compute the Jaccard similarity between all pairs of documents for each type of k-gram

Length of all files:

```
D1 file has length: 1749
D2 file has length: 1747
D3 file has length: 2132
D4 file has length: 1435
```

Analysis:

- **Character 2-Gram**:
  - D1 and D2 show extremely high similarity (0.98), indicating almost identical content.
  - D1–D3 and D2–D3 also show strong similarity (~0.8).
  - D4 has comparatively lower similarity with other documents.
- **Character 3-Gram**:
  - D1 and D2 remain highly similar (0.977).
  - The similarity between D3/D4 and the other documents (D1 and D2) decreases significantly, indicating that longer n-grams impose stricter matching.
- **Word 2-Gram**:
  - D1 and D2 still remain highly similar (0.94).
  - Similarity among other document pairs reduces drastically, showing that word-level matching is more restrictive.

```
********************** Question 1 Create k-Gram **********************
=== Character 2-grams ===
D1 D2 Jaccard Value for 2-grams char:  0.9811320754716981
D1 D3 Jaccard Value for 2-grams char:  0.8156996587030717
D1 D4 Jaccard Value for 2-grams char:  0.6444444444444445
D2 D3 Jaccard Value for 2-grams char:  0.8
D2 D4 Jaccard Value for 2-grams char:  0.6412698412698413
D3 D4 Jaccard Value for 2-grams char:  0.6529968454258676

=== Character 3-grams ===
D1 D2 Jaccard Value for 3-gram char:  0.977979274611399
D1 D3 Jaccard Value for 3-gram char:  0.5803571428571429
D1 D4 Jaccard Value for 3-gram char:  0.3050847457627119
D2 D3 Jaccard Value for 3-gram char:  0.5680473372781065
D2 D4 Jaccard Value for 3-gram char:  0.30590339892665475
D3 D4 Jaccard Value for 3-gram char:  0.31212381771281167

=== Word 2-grams ===
D1 D2 Jaccard Value for word 2-gram:  0.9407665505226481
D1 D3 Jaccard Value for word 2-gram:  0.18234165067178504
D1 D4 Jaccard Value for word 2-gram:  0.03024193548387097
D2 D3 Jaccard Value for word 2-gram:  0.1736641221374046
D2 D4 Jaccard Value for word 2-gram:  0.030303030303030304
D3 D4 Jaccard Value for word 2-gram:  0.01607142857142857
```

Question 2: Min-Hashing:

Analysis:

1. For small **t** values (**20–300**), the estimate slightly overestimates the similarity. (e.g., 1.0, 0.9833, 0.9933).
2. At **t = 600**, the estimate stabilizes closer to the exact Jaccard similarity i.e. **0.9733** (nearest to Exact Similarity = 0.9780)
3. Increasing t reduces the variance of the Min-Hash estimate
4. Larger t values provide more stable and reliable approximations, although they increase computational cost

t = 600 is the best choice for accuracy because it gives an estimate closest to the exact Jaccard similarity.

```
*********** Question 2 Min-Hashing ************
Exact similarity: 0.977979274611399
t = 20 Estimated similarity: 1.0
t = 60 Estimated similarity: 0.9833333333333333
t = 150 Estimated similarity: 0.9933333333333333
t = 300 Estimated similarity: 0.9833333333333333
t = 600 Estimated similarity: 0.9733333333333334
```

Question 3: LSH Probabilities

Analysis:

- Using r = 8 b = 20:
    - D1 & D2 Probability: 0.9999999999999998 ≈ 1.0
    - D1 & D3 Probability: 0.22822420166613133
    - D1 & D4 Probability: 0.001499976039931461
    - D2 & D3 Probability: 0.1958804369207785
    - D2 & D4 Probability: 0.0015324562526581875
    - D3 & D4 Probability: 0.0018000048667899948
- The probability for D1 & D2 is ~ 1.0, meaning they will almost always be placed in the same bucket.
- All other document pairs have significantly lower probabilities (close to zero in most cases), showing that LSH effectively separates highly similar and dissimilar document pairs.

```
...  ****************** Question 3 LSH Probabilities ************
     Using r = 8 b = 20
     D1 & D2 Probability: 0.9999999999999998
     D1 & D3 Probability: 0.22822420166613133
     D1 & D4 Probability: 0.001499976039931461
     D2 & D3 Probability: 0.1958804369207785
     D2 & D4 Probability: 0.0015324562526581875
     D3 & D4 Probability: 0.0018000048667899948
```

Question 4: Min-Hashing on MovieLens Dataset

Analysis:

- **Average errors**:
    - **t=50**
        - AVERAGE FALSE POSITVES for t = 50: **156.8**
        - AVERAGE FALSE NEGATIVES for t = 50: **1.6**
    - **t=100**
        - AVERAGE FALSE POSITVES for t = 100: **24.6**
        - AVERAGE FALSE NEGATIVES for t = 100: **2.8**
    - **t=200**
        - AVERAGE FALSE POSITVES for t = 200: **9.8**
        - AVERAGE FALSE NEGATIVES for t = 200: **2.4**

- At **t=50**, the number of false positives is high, meaning the algorithm detects many extra pairs.
- As the number of hash functions increases (**t=100 and t=200**), false positives decrease significantly, indicating better filtering.
- False negatives fluctuate slightly but remain relatively small. Overall, increasing the signature size reduces variance and improves reliability.

```
print("Total number of such pairs:", len(exact))
```

[12] ✓ 4.0s

```
User pairs with similarity >= 0.5:
(197, 600)
(197, 826)
(328, 788)
(408, 898)
(451, 489)
(489, 587)
(554, 764)
(600, 826)
(674, 879)
(800, 879)
Total number of such pairs: 10
```

[13] ✓ 25.6s

```
t=50, pairs ≥0.5: 137
t=100, pairs ≥0.5: 51
t=200, pairs ≥0.5: 11
```

[16] ✓ 2m 22.6s

```
********** Question 4: Min-Hashing on MovieLens Dataset **********

--- Running 5 trials for t=50 hash functions ---
  Run 1/5 => False Positives: 72, False Negatives: 2
  Run 2/5 => False Positives: 221, False Negatives: 1
  Run 3/5 => False Positives: 31, False Negatives: 3
  Run 4/5 => False Positives: 130, False Negatives: 1
  Run 5/5 => False Positives: 330, False Negatives: 1
-------------------------------------
AVERAGE FALSE POSITVES for t = 50: 156.8
AVERAGE FALSE NEGATIVES for t = 50: 1.6
-------------------------------------

--- Running 5 trials for t=100 hash functions ---
  Run 1/5 => False Positives: 46, False Negatives: 1
  Run 2/5 => False Positives: 24, False Negatives: 4
  Run 3/5 => False Positives: 26, False Negatives: 2
  Run 4/5 => False Positives: 14, False Negatives: 2
  Run 5/5 => False Positives: 13, False Negatives: 5
-------------------------------------
AVERAGE FALSE POSITVES for t = 100: 24.6
AVERAGE FALSE NEGATIVES for t = 100: 2.8
-------------------------------------
```

```
--- Running 5 trials for t=200 hash functions ---
  Run 1/5 => False Positives: 7, False Negatives: 5
  Run 2/5 => False Positives: 8, False Negatives: 1
  Run 3/5 => False Positives: 10, False Negatives: 2
  Run 4/5 => False Positives: 14, False Negatives: 2
  Run 5/5 => False Positives: 10, False Negatives: 2
------------------------------------
AVERAGE FALSE POSITVES for t = 200: 9.8
AVERAGE FALSE NEGATIVES for t = 200: 2.4
------------------------------------
```

Question 5: LSH on MovieLens dataset

Analysis:

Effect of Increasing the Similarity Threshold from 0.6 to 0.8

- **Threshold = 0.6**
  - False Positives is high
  - False Negatives is ~ 0
- **Threshold =0.8**
  - False Positives is high
  - False Negatives ~ 0

- When the threshold is increased from 0.6 to 0.8, the similarity condition becomes stricter. Ideally, this should reduce false positives because fewer pairs qualify as similar.
- However, in our results (with r = 5), false positives remain high even at threshold 0.8. This indicates that band configuration has a stronger impact on error rates than the threshold alone.
- When r is increased to 10 (with 200 hashes), false positives drop dramatically to 2, showing that band size plays a crucial role in controlling incorrect matches.

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ...

[24] ✓ 1.8s

```
LSH candidates (50 hashes, r=5): 533
```

```python
sig100 = minhash_users(100)

candidates_100 = lsh(sig100, r=5)

print("LSH candidates (100 hashes, r=5):", len(candidates_100))
```

[25] ✓ 3.5s

```
LSH candidates (100 hashes, r=5): 1540
```

```python
sig200 = minhash_users(200)

candidates_200_a = lsh(sig200, r=5)

print("LSH candidates (200 hashes, r=5):", len(candidates_200_a))
```

[19] ✓ 6.4s

```
LSH candidates (200 hashes, r=5): 1544
```

```python
candidates_200_b = lsh(sig200, r=10)

print("LSH candidates (200 hashes, r=10):", len(candidates_200_b))
```

[20] ✓ 0.0s

```
LSH candidates (200 hashes, r=10): 3
```

```
********* Question 5 LSH on MovieLens Dataset **************

Threshold = 0.6, r = 5
Avg False Positives: 406.0
Avg False Negatives: 2.0

Threshold = 0.8, r = 5
Avg False Positives: 406.0
Avg False Negatives: 0.0

Threshold = 0.6, r = 5
Avg False Positives: 925.0
Avg False Negatives: 0.0

Threshold = 0.8, r = 5
Avg False Positives: 927.0
Avg False Negatives: 0.0

Threshold = 0.6, r = 5
Avg False Positives: 1799.0
Avg False Negatives: 0.0

Threshold = 0.8, r = 5
Avg False Positives: 1801.0
Avg False Negatives: 0.0

Threshold = 0.6, r = 10
Avg False Positives: 1.0
Avg False Negatives: 2.0
```

```
Threshold = 0.8, r = 10
Avg False Positives: 1.0
Avg False Negatives: 0.0
```

```python
import pandas as pd
df_results = pd.DataFrame(results)
df_results.sort_values(by=["Configuration", "Threshold"])
```

[37] ✓ 0.0s

	Configuration	Threshold	r	Avg False Positives	Avg False Negatives
2	100 hashes	0.6	5	1171.0	1.0
3	100 hashes	0.8	5	1172.0	0.0
4	200 hashes	0.6	5	2185.0	0.0
6	200 hashes	0.6	10	2.0	3.0
5	200 hashes	0.8	5	2187.0	0.0
7	200 hashes	0.8	10	2.0	1.0
0	50 hashes	0.6	5	362.0	1.0
1	50 hashes	0.8	5	363.0	0.0